

BCSE301L_SOFTWARE ENGINEERING



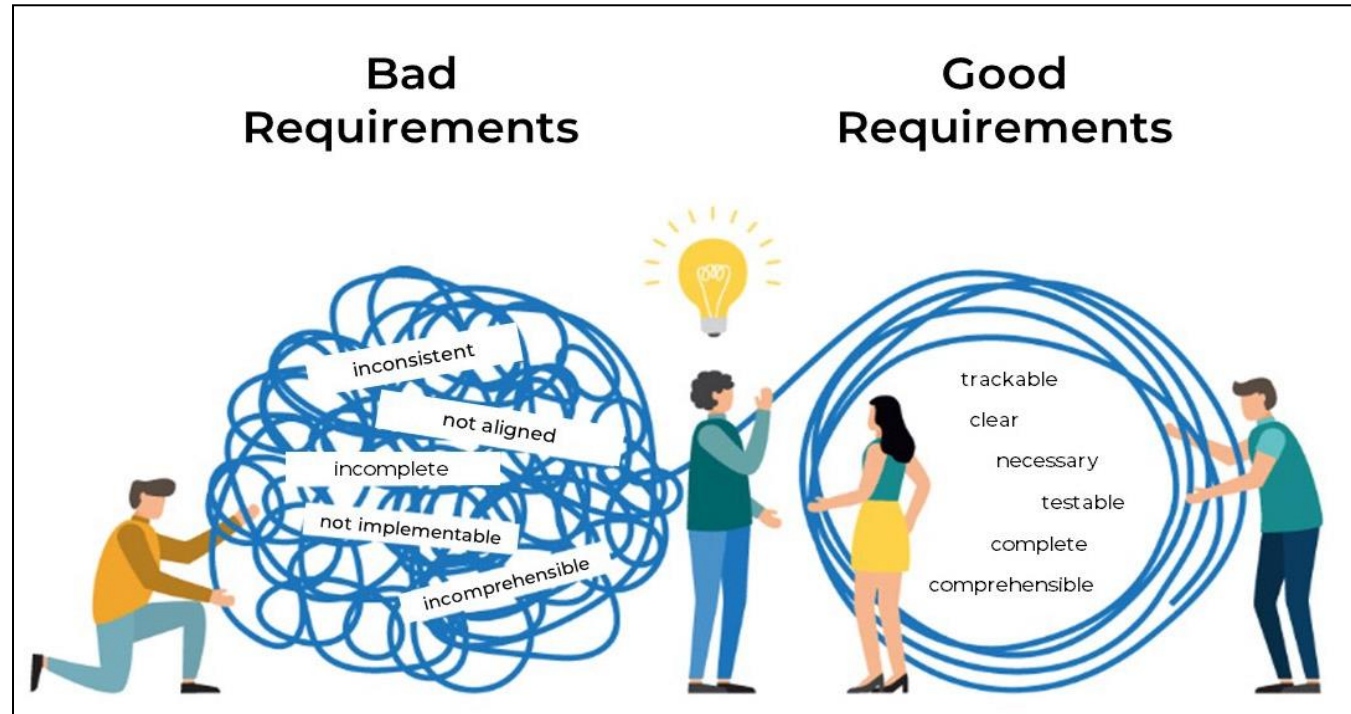
MODELLING REQUIREMENTS

Prepared by Dr. Baiju B V, SCOPE

Presented By,
Dr. Baiju B V
Assistant Professor
SCOPE, VIT, Vellore

SOFTWARE REQUIREMENTS AND ITS TYPES

- Requirements are *descriptions of the services* that a software system must provide and the constraints under which it must operate.
- Requirements can *range from high-level abstract statements of services or system constraints to detailed mathematical functional specifications.*
- The requirements can be **clear or hidden, known or unknown, expected or unexpected** from client's point of view.

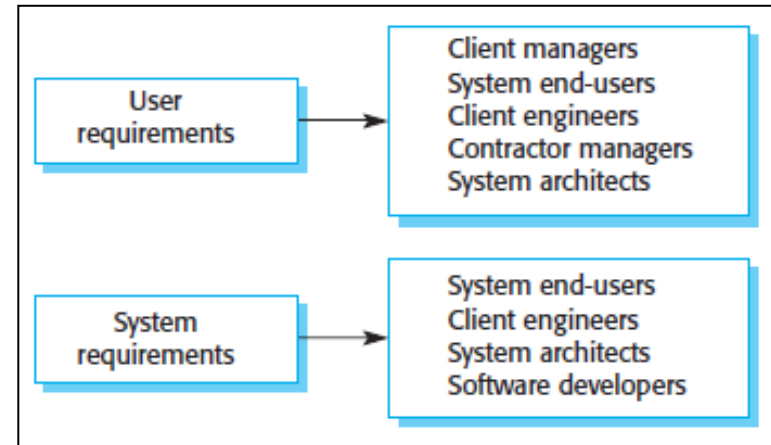


- Requirements engineering (RE) is the process of **finding out, analyzing, documenting** and **checking these services and constraints**.
- Requirement engineering provides the appropriate mechanism to understand
 - *Customer desires / wishes*
 - *Analysing customer needs*
 - *Assessing feasibility*
 - *Negotiating a reasonable solution*
 - *Specifying the solution clearly*
 - *Validating the specification and managing the requirement*
- Some of the problems that arise during the requirements engineering process are a result of failing to make a clear separation between these different levels of description.
- It can be differentiated by using the term **user requirements** to mean the **high-level abstract requirements** and **system requirements** to mean the **detailed description** of what the system should do.

- User requirements and System requirements may be defined as follows:

User Requirements	System Requirements
Written for customers	Written for implementation team
Statements in natural language	Statements in technical language
Describe the services and features provided by system	Describe the detailed description of services, features and complete operations of system
May include diagrams and tables	May include system models
Understandable by system users who don't have technical knowledge	Understand by implementation team who have technical knowledge

- The readers of the system requirements need to know more precisely what the system will do because they are concerned with how it will support the business processes or because they are involved in the system implementation
- Readers of different types of requirements specification



User requirements definition

1. The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2 The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
- 1.5 Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Functional and Non-functional requirements

- Software system requirements are often classified as functional or non-functional requirements:
- 1. Functional requirements**
- The functional requirements for a system describe *what the system should do*.
- These requirements depend on the
 - *type of software being developed*
 - *the expected users of the software*
 - *the general approach taken by the organization when writing requirements.*

System: An Online Shopping Platform

Functional Requirement:

- The system must allow registered users to search for products by name, category, or brand and display relevant search results within 2 seconds.

What: The system provides a product search functionality.

Who: This feature is accessible to registered users.

How: Users can search using specific parameters like name, category, or brand.

Behavior: The system must fetch and display results promptly (within 2 seconds).

2. Non-functional requirements

- Non-functional requirements are requirements that are *not directly concerned with the specific services delivered by the system to its users*.
- These non-functional requirements usually specify the constrain characteristics of the system as a whole.

Performance: The system should process transactions within 2 seconds.

Security: User data must be encrypted both in transit and at rest.

Usability: The system should be accessible to users with disabilities, complying with WCAG 2.1 standards.

Reliability: The system should have an uptime of 99.9% over a year.

Scalability: The system should support up to 10,000 concurrent users without performance degradation.

FUNCTIONAL REQUIREMENTS

WHAT THE SYSTEM SHOULD DO



PRODUCT FEATURES



USER REQUIREMENTS

NON-FUNCTIONAL REQUIREMENTS

HOW THE SYSTEM SHOULD DO IT



PRODUCT PROPERTIES

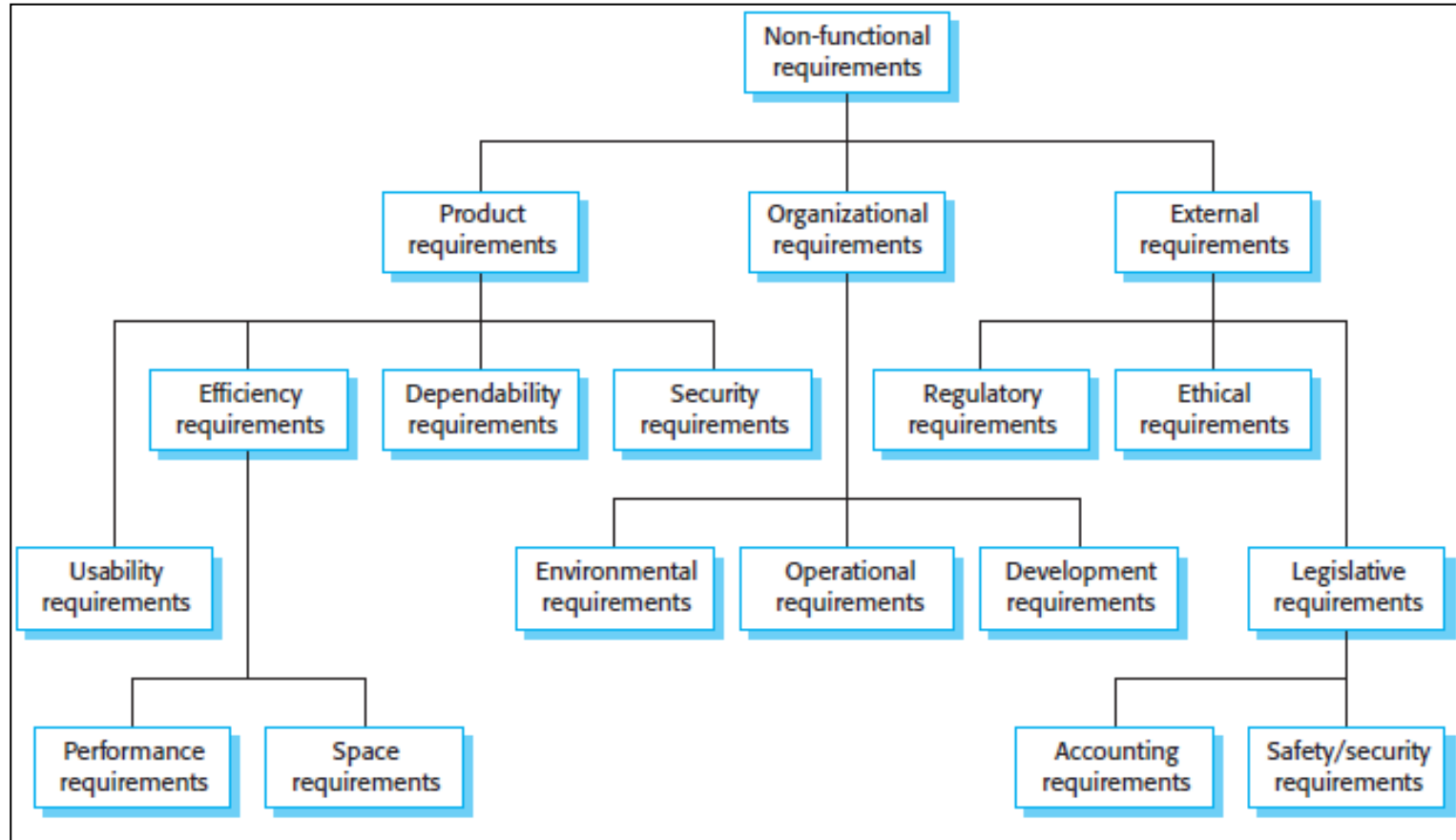


USER EXPECTATIONS

S.No.	Functional Requirements	Non-Functional Requirements
1	A functional requirement represents software and its elements and features.	It represents the quality characteristic of a system.
2	The functional requirements mentioned the activities a software system should perform.	It sets some rules and regulations for a software system on how it should fulfill the essential need.
3	These requirements are established by the user.	These requirements are established by technical persons, such as e.g. software developers, architects, and more.
4	These requirements are compulsory.	These are not compulsory.
5	These are easy to define.	These are hard to define.
6	Allow you to measure the functionality of the software.	Allows you to check the performance of the system.
7	System, Integration, End to End, API testing, etc are functional testing.	Performance, Stress, Usability, Security testing, etc are non-functional testing.
8	These requirements are vital to system operation.	These are not always vital requirements.

Types of non-functional requirements

- Non-functional requirements may come from *required characteristics of the software* (product requirements), the *organization developing the software* (organizational requirements), or *external sources*.



Product requirements

- These requirements *specify or constrain the runtime behavior of the software*.
- Examples include
 - *Performance requirements for how fast the system must execute*
 - *How much memory it requires*
 - *Reliability requirements that set out the acceptable failure rate*
 - *Security requirements*
 - *Usability requirements.*

Performance: The website should load within 2 seconds for 90% of users..

Memory: The mobile application should not consume more than 50 MB of RAM during normal operation.

Reliability: The system must not fail more than once per 10,000 transactions.

Security: User passwords must be encrypted using AES-256.

Usability: A first-time user should be able to complete the registration process within 2 minutes without any assistance.

Organizational requirements

- These requirements are broad system requirements *derived from policies and procedures in the customer's and developer's organizations.*
- Examples include
 - **Operational process** requirements that define how the system will be used
 - **Development process** requirements that specify the programming language, development environment or process standards to be used
 - **Environmental requirements** that specify the operating environment of the system.

Operational Process Requirements: The system should support 24/7 operations with automated backup every 12 hours.

Development Process Requirements: The software must be developed using Python and adhere to Agile development methodologies.

Environmental Requirements: The system should operate in temperatures ranging from 10°C to 35°C and be compatible with Windows and Linux operating systems.

External requirements

- This broad heading covers all requirements that are *derived from factors external to the system and its development process*.
- Example include
 - ***Regulatory requirements** that set out what must be done for the system to be approved for use by a regulator, such as a nuclear safety authority*
 - ***Legislative requirements** that must be followed to ensure that the system operates within the law.*
 - ***Ethical requirements** that ensure that the system will be acceptable to its users and the general public.*

Regulatory Requirements: The system must comply with GDPR for data protection and privacy.

Legislative Requirements: The software must adhere to copyright laws to avoid legal issues.

Ethical Requirements: The system should not collect or use user data without explicit consent.

Metrics for specifying non-functional requirements

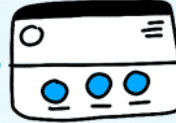
Property	Measure
<i>Speed</i>	Processed transactions/second User/event response time Screen refresh time
<i>Size</i>	Megabytes/Number of ROM chips
<i>Ease of use</i>	Training time Number of help frames
<i>Reliability</i>	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
<i>Robustness</i>	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
<i>Portability</i>	Percentage of target dependent statements Number of target systems

FUNCTIONAL REQUIREMENTS

examples of

NON-FUNCTIONAL REQUIREMENTS

the website will have a homepage



the homepage should load within 1.5 seconds

the website will store customer data



customer data will be encrypted to level 4 encryption standards

website customers can log into their accounts



the website has the capacity for 5,000 logged-in users at any one time

the website will always be available to customers



the website must have a 99.7% uptime

customers can access the websites on their phones



the website will be compatible with iOS 12.8 and above

FUNCTIONAL REQUIREMENTS

VS

NON-FUNCTIONAL REQUIREMENTS

The vehicle must be fast



The engine in the vehicle should be able to reach at least 150 MPH

The vehicle must be red



The vehicle should be painted using House of Kolor - KBC11 - Apple Red

The vehicle must be safe for drivers



The vehicle and all its parts should comply with all Federal Motor Vehicle Safety Standards (FMVSS)

Rear camera in the automobile should detect a threat or object

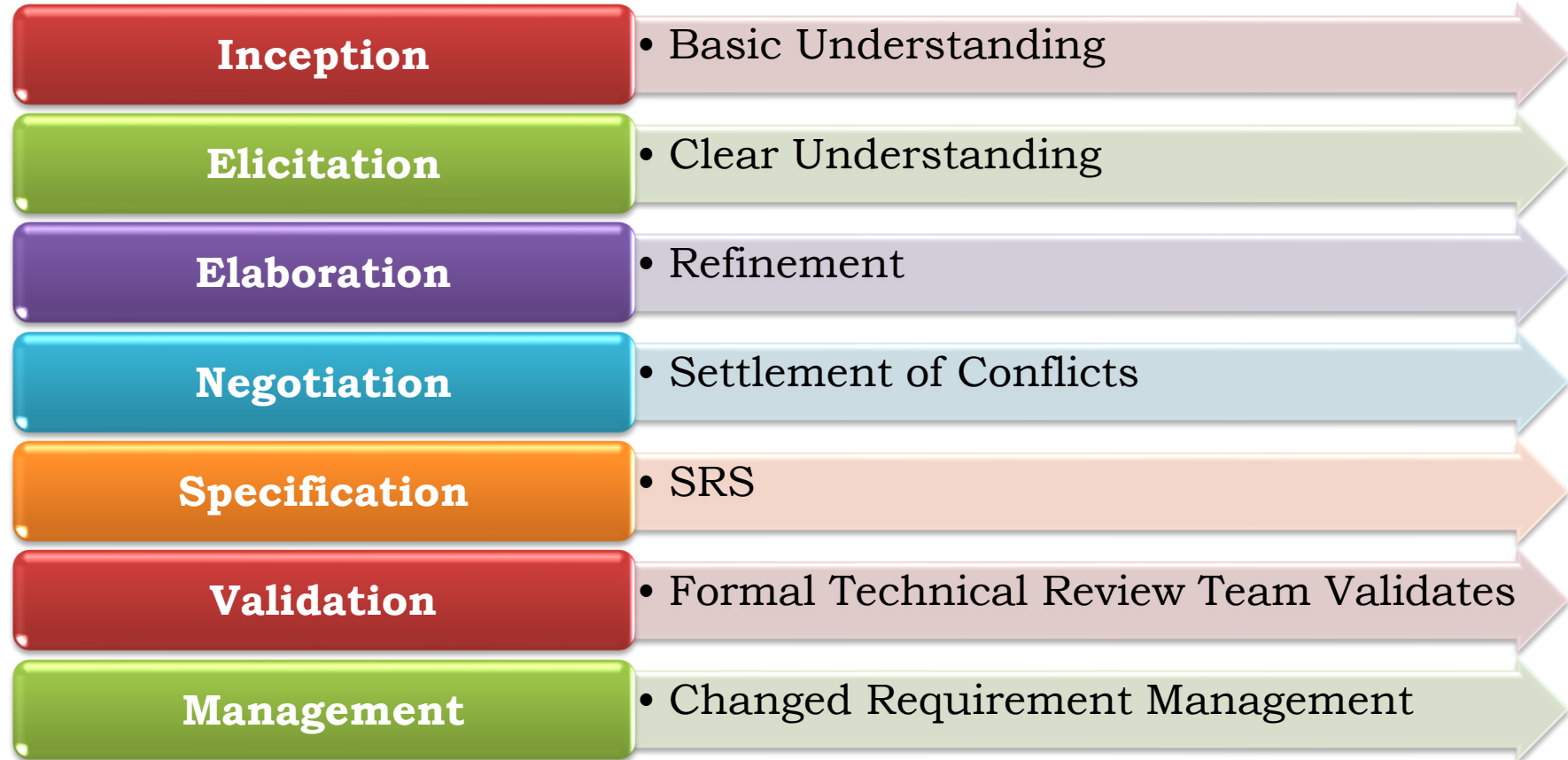


Rear camera must notify the driver of a threat or object within .2 seconds of the detection

REQUIREMENTS ENGINEERING PROCESSES

- Requirements engineering in software engineering is a **critical** and **initial phase of project development**, essential for successful systems.
- Requirements engineering is the term for the broad spectrum of tasks and techniques that lead to an understanding of requirements
- It serves as a **bridge between stakeholders' objectives and software possibilities**, shaping **project scope**, **schedule**, and **outcomes**.
- Requirements engineering involves three key activities.
 1. **Elicitation and Analysis** : Discovering requirements by interacting with stakeholders
 2. **Specification**: Converting these requirements into a standard form
 3. **Validation**: Checking that the requirements actually define the system that the customer wants
- Requirements engineering is an **iterative process** in which the activities are interleaved.
- Requirements engineering **builds a bridge to design and construction**.

- Requirements engineering encompasses seven tasks with sometimes boundaries:



1. Inception

- This phase gives an *outline of how to get started on a project*.
- Communication between all stakeholders and the software team needs to be established during this task to begin an effective collaboration.
- During inception, the requirements engineer asks a set of questions to establish
 - *A basic understanding of the problem*
 - *The people who want a solution*
 - *The nature of the solution that is desired*
 - *The effectiveness of preliminary communication and collaboration between the customer and the developer*
- Set of questions
 - Who is behind the request for this work?
 - Who will use the solution?
 - What will be the economic benefit of a successful solution?
 - Is there another source for the solution that you need?
 - Are my questions relevant to the problem that you have?
 - Am I asking too many questions?
 - Can anyone else provide additional information?

2. Elicitation

- This phase focuses on *gathering the requirements from the stakeholders*.
- The *right people* must be involved in this phase, no space for mistake.
- The requirements are difficult because the following problems occur in elicitation.

Problem of Scope:

- The requirements given are of **unnecessary detail**, **unclear**, or **not possible to implement**.

Problem of Understanding:

- **Not having a clear understanding between the developer and customer**, when putting out the requirements needed.
- Sometimes the customer might not know what they want or the developer might misunderstand one requirement for another.

Problem of Volatility:

- **Requirements changing over time** can cause difficulty in leading a project.
- It can lead to loss and wastage of resources and time.

3. Elaboration

- This phase is the *result of the inception and elicitation phase*.
- It takes the requirements that have been stated and gathered in the first two phases and refines them.
- The main task in this phase is to **indulge in modeling activities** and **develop a prototype** that elaborates on the features and constraints using the necessary tools and functions.

4. Negotiation

- Negotiation is between the developer and the customer and they dwell on how to go about the project with limited business resources.
- Customers are asked to prioritize the requirements for development and analyze any conflict has occur.
- The following are discussed in the negotiation phase:
 - *Availability of Resources.*
 - *Delivery Time.*
 - *Scope of requirements.*
 - *Project Cost.*
 - *Estimations on development.*

5. Specification

- In the specification phase, the *requirements engineer gathers all the requirements and develops a working model.*
- This phase specifies the following:
 - *Written document*
 - *A set of models*
 - *A collection of use cases*
 - *A prototype*
- The models used in this phase include
 - *ER (Entity Relationship) diagrams*
 - *DFD (Data Flow Diagram)*
 - *FDD (Function Decomposition Diagrams)*
 - *Data Dictionaries.*
- A software specification document is submitted to the customer in a language that he/she will understand, to give a glimpse of the working model.

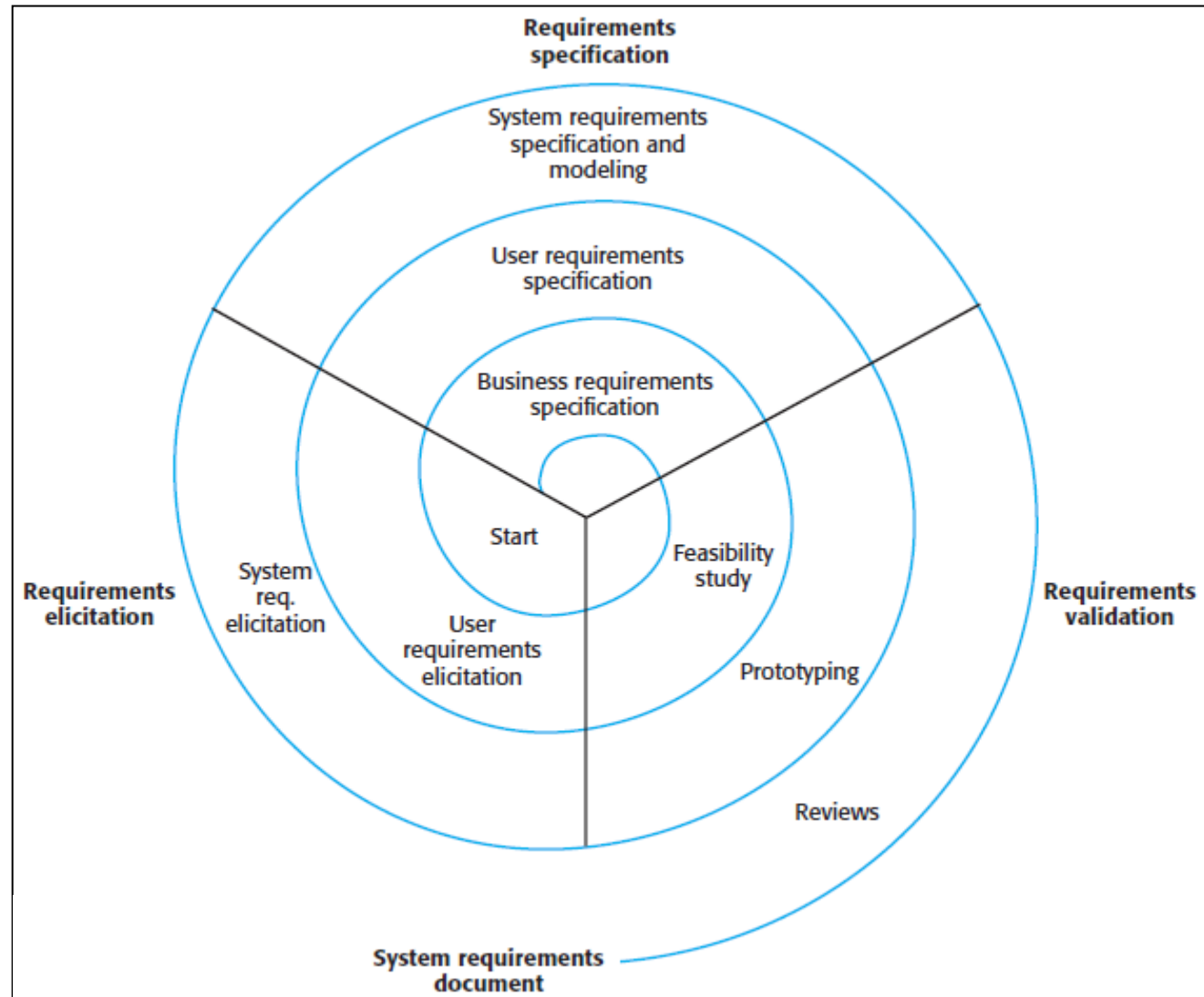
6. Validation

- In the validation phase, *the developer scans the specification document* and checks for the following:
 - All the requirements have been stated and met correctly
 - Errors have been debugged and corrected.
 - Work product is built according to the standards.
- The **formal technical reviews** from the software engineer, customer and other stakeholders helps for the primary requirements validation mechanism.

7. Requirement Management

- It is a set of activities that help the project team to identify, control and track the requirements and changes can be made to the requirements at any time of the ongoing project.
- It *track on new additional requirement implementation* which does not affect on overall system.
- Based on this phase, the working model will be analyzed carefully and ready to be delivered to the customer.

- The activities are organized as an **iterative process** around a spiral.
- **The output of the RE process is a system requirements document.**
- The amount of time and effort devoted to each activity in an iteration depends on the stage of the overall process, the type of system being developed, and the budget that is available.



- Early in the process, most effort will be spent on understanding high-level business and non-functional requirements, and the user requirements for the system.
- Later in the process, in the outer rings of the spiral, more effort will be devoted to eliciting and understanding the non-functional requirements and more detailed system requirements.
- This spiral model accommodates approaches to development where the requirements are developed to different levels of detail.
- The number of iterations around the spiral can vary so that the spiral can be exited after some or all of the user requirements have been elicited.
- Agile development can be used instead of prototyping so that the requirements and the system implementation are developed together.
- The people involved develop a better understanding of
 - *what they want the software to do*
 - *the organization buying the system changes*
 - *modifications are made to the system's hardware, software, and organizational environment.*

REQUIREMENT ELICITATION

- Requirements elicitation is the *process of gathering and defining the requirements* for a software system.
- The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.
- During requirements elicitation, software engineers work with stakeholders to find out about
 - *Application domain*
 - *Work activities*
 - *Services and system features that stakeholders want*
 - *Required performance of the system*
 - *Hardware constraints*

REQUIREMENT ELICITATION

Challenges of Requirements Elicitation

As an analyst, I
need to know what
do you want?



I want you to
design the
software for me.



But what do you
want to do with
the software?



I don't know until
you tell me what the
software can do.



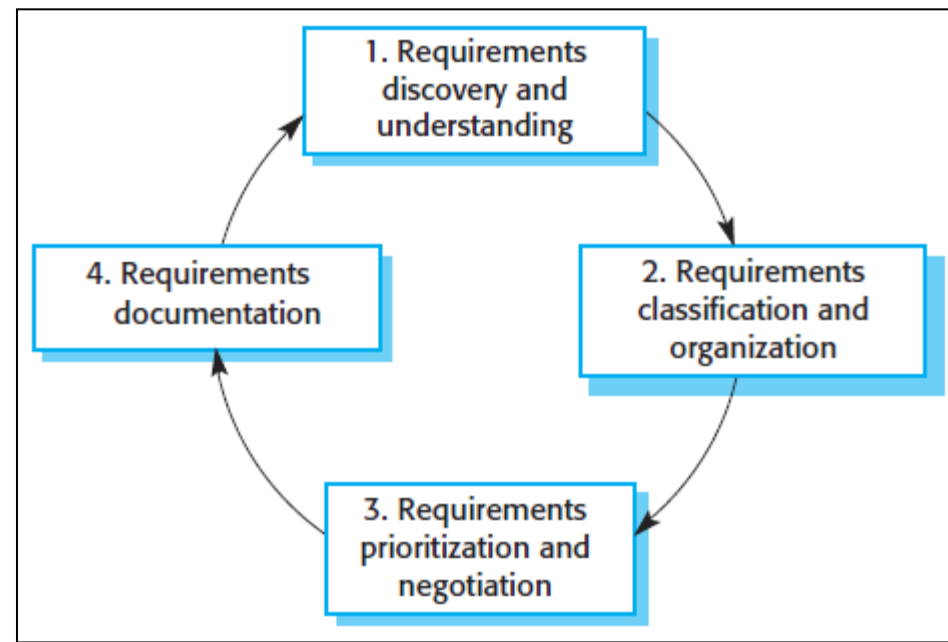
Well, I can design
the software to do
anything!



Can you design the
software to tell you
my requirements?!



- Each organization will have its own version of this general model, depending on local factors such as
 - *Expertise of the staff*
 - *Type of system being developed*
 - *Standards used.*
- The process activities are:



a. Requirements discovery and understanding

- This is the **process of interacting with stakeholders** of the system to discover their requirements.
- Domain requirements from stakeholders and documentation are also discovered during this activity.

b. Requirements classification and organization

- This activity takes the **unstructured collection of requirements, groups related requirements** and organizes them into coherent clusters.

3. Requirements Prioritization and Negotiation

- When **multiple stakeholders** are involved, requirements will conflict.
- This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation.
- Stakeholders have to meet to resolve differences and agree on compromise requirements.

4. Requirements Documentation

- The **requirements are documented and input into the next round of the spiral**.
- An early draft of the software requirements documents may be produced at this stage, or the requirements may simply be maintained informally on whiteboards, wikis, or other shared spaces.

-
- Requirements elicitation and analysis is an iterative process with continual feedback from each activity to other activities.
 - The process cycle starts with requirements discovery and ends with the requirements documentation.
 - The analyst's understanding of the requirements improves with each round of the cycle. The cycle ends when the requirements document has been produced.